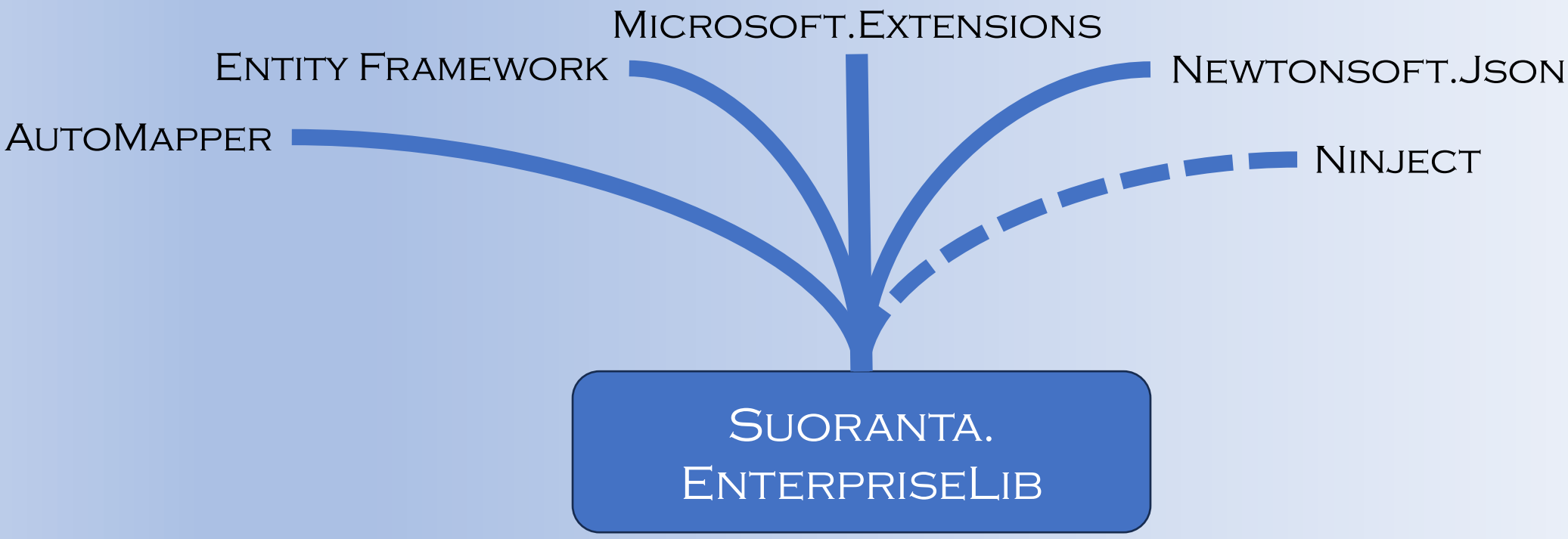
Building & Salvaging Data-centric (ASP).NET Apps With

Suoranta.EnterpriseLib



Robust & Maintainable Enterprise Applications

Document Version 11/2026 By Lauri Suoranta

If Surgeons were like Software Engineers…

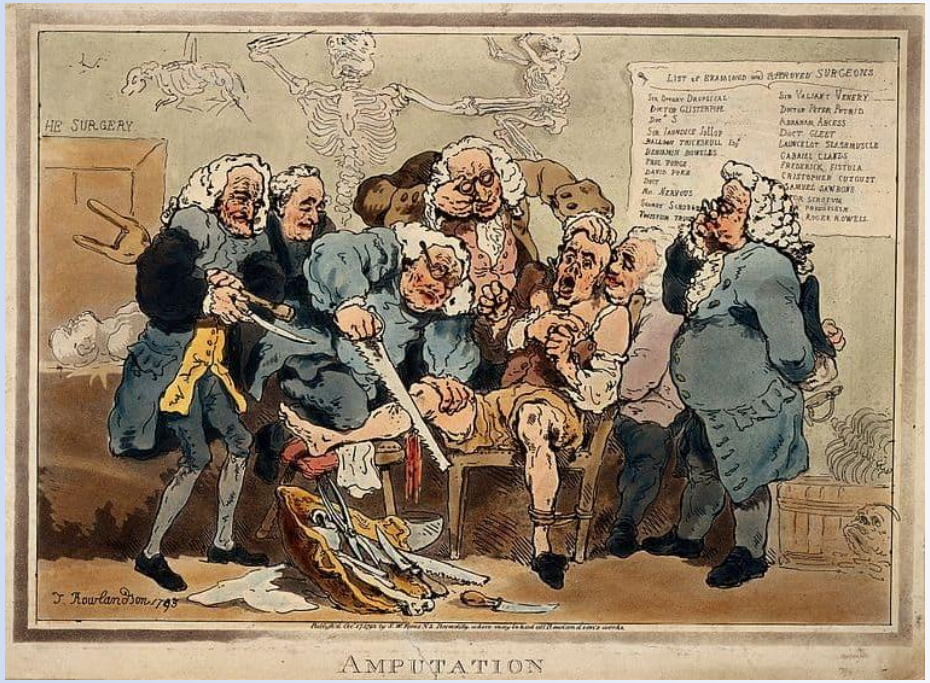
(and they were — before the late 19th century)

No formal background required to start practicing.

What one surgeon calls the spleen is the liver to the other...[1]

Antiseptic practices according to taste of the practitioner.[2]

”Speed is of the essence…”



Amputation by cartoonist Thomas Rowlandson.

[1] Person can live without a spleen. Splenectomy is sometimes life-saving. Humans cannot live without a liver.

[2] Antiseptic practices were invented by Lister in 1865, but they were contested for several decades.

EnterpriseLib in a Nutshell



EnterpriseLib.Base

Shared interfaces

Utility methods

XML-based configuration system[1] with full schema

EnterpriseLib

DTO-to-LINQ

Logging facilities[2]

Mock data toolkit

EnterpriseLib.Web

Base controllers (MVC, WebAPI)

Dependency injector

+ auto-configurators

Entity-DTO-Entity auto-mapping

HTTP server middleware

Push notification service

.Net Framework (MVC 5) only

[1] ”Legacy” System.Configuration. Not a concern as XML configurations can also be read by Microsoft.Extensions.

[2] Implements Microsoft.Extensions.Logging as a sidenote. The primary functionality is much better.

.Net Framework vs. .NET

“ELib” capabilities that are not strictly tied to ASP.NET are agnostic of .Net/.NET flavor. ELib.Web also has a range of ASP.NET capabilities that are not specifically mentioned[1] (except MVC Dependency Resolver).

ASP.NET MVC 5 (.Net Framework) vs. ASP.NET Core (.NET):

- Existing ASP.NET MVC 5 apps widely in use and will require servicing into 2030s. Quality of architecture highly variable. ELib.Web ideal for salvaging such apps that lack solid architecture.
- The Core MVC app hosting model improved, which has made certain ELib.Web features less relevant or obsolete. On the other hand, ELib on MVC 5 is in tune with the approach adopted by Core, giving the option to migrate MVC 5 apps.

[1] The discussion of these features would have to include excessive amount of technical details.

Features — Some Highlights

DTO-to-LINQ, or Query Objects:

- Create LINQ queries declaratively, by decorating DTO[1] properties with attributes that describe their use in the query. ELib transforms the object into LINQ. Can be combined with developer's own LINQ before executed against the database.

Entity-DTO-Entity Transforms:

- AutoMapper extensions that address the lack of entity tracking across web requests. An object graph with multiple added or updated entities can be pushed back to the database on single request/transaction.

More in API Documentation and Examples: Suoranta.Github.io/ApiDocs/

[1] DTO, data transfer object, represents here query options that an end-user has entered into a form in the application.

Dependency Injection in EnterpriseLib



In .Net Framework:

ELib can auto-configure Microsoft.Extensions DI and Ninject (for the common use cases). It can also forward requests between the two.

- Forwarding is ideal for migrating from Ninject to Microsoft.Extensions (the latter seems now more widely supported).

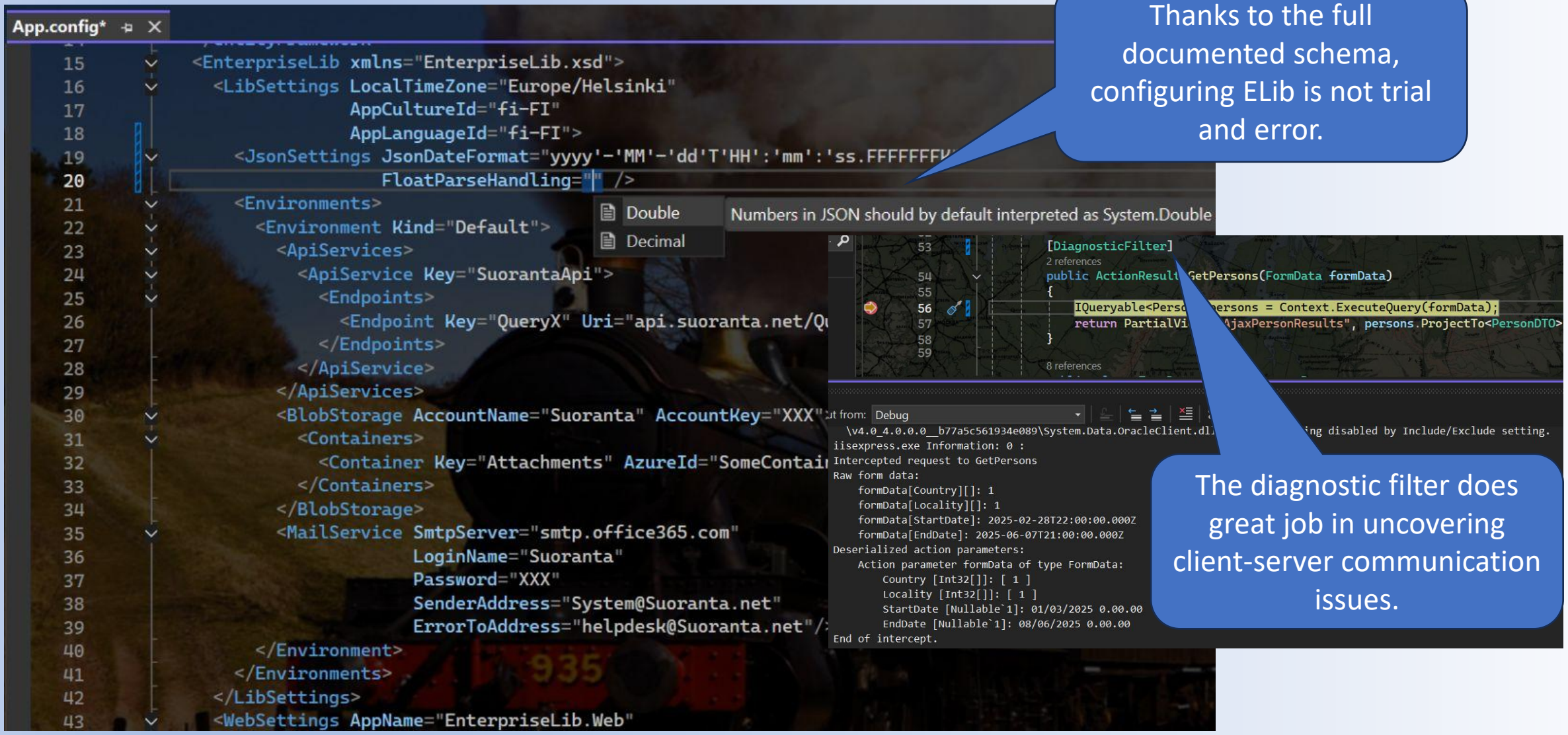
In MVC applications, ELib deploys its own MVC `DependencyResolver` that can operate either in exclusive or dual mode.

- The underlying `ServiceProvider` and/or Ninject `IKernel` still accessible.

In ASP.NET "Core" (.NET 5+):

- Vanilla Microsoft.Extensions works great. "Don't fix what's not broken." ELib will only provide extension methods to access the auto-configurator modules.

Convenience Features — Examples



Thanks to the full documented schema, configuring ELib is not trial and error.

The diagnostic filter does great job in uncovering client-server communication issues.

Deployment

Deployed through private NuGet repository (three EnterpriseLib, several proprietary support packages). Setup:

- Through custom configuration classes that ELib locates automatically from search assemblies (this can be fine-tuned, of course).
- Extensions methods for fluent/builder style.[1]
- XML (web.config in .Net Framework).

Interface-to-class mapping, object instantiation and lifecycle fully managed by DI. Allows any bit of ELib behavior to be overridden in exotic use cases, without throwing away any of the rest.

[1] Configuration classes are largely fluent, but syntax is of base infrastructure rather than of EnterpriseLib. EnterpriseLib cuts away the slack. Proprietary fluent is used more extensively in **Suoranta.WebApp, the front-end counterpart**.

Conclusion

EnterpriseLib has been designed keeping in mind that the greatest needs are in the large existing application base. Features can be taken into use gradually, without refactoring the whole application.

ELib relies on **.NET reflection** to map the structure of target app. This has some limitations:

- Incoherent database structure, such as incomplete linkage between entities, or lack of naming convention, can limit the utility of AutoMapper-related functionality (more manual configuration is needed).
- Refactoring that does not require database schema changes is generally straightforward. Where such refactoring is required, it would almost certainly be advisable even without ELib.